Management Summary: Multilingual Language Identification

January 13, 2026

1 Overview of the Task

Our project focuses on identifying the language of short text snippets drawn from multilingual Wikipedia. Each snippet belongs to one of ten target languages across both Latin and Cyrillic scripts (e.g., English, German, French, Urdu, Kazakh). The goal is to automatically recognize the language of a sentence so downstream systems—search, content moderation, or localization workflows—can route content correctly.

2 Key Challenges

Several languages share alphabets and common vocabulary, which makes them hard to distinguish at the sentence level. Some inputs are very short, contain names or numbers, or borrow words from other languages, all of which reduce the number of reliable clues. We also worked with multiple scripts (Latin vs. Cyrillic), which can introduce transliteration and encoding edge cases.

3 External Resources Used

We used the multilingual Wikipedia data provided for the course and processed it with Stanza for sentence segmentation. For modeling, we relied on standard, open-source tools: scikit-learn for the lightweight machine learning baseline, and the Hugging Face Transformers library for the multilingual XLM-R model. These resources allowed us to compare interpretable baselines with a modern transformer approach.

4 Solution Implemented

We evaluated three approaches and selected the most effective and practical one as our primary solution. First, we built a rule-based heuristic system that checks scripts, diacritics, and hand-curated cue words. Second, we trained a character n-gram logistic regression model, which learns statistical patterns in spelling and is computationally cheap. Third, we fine-tuned the multilingual transformer XLM-R for sequence classification. The character n-gram model delivered the highest accuracy (about 97%) while remaining easy to train and deploy, so it is the recommended choice for the project's final submission.

5 Final Solution (fastText Primary + Character n-gram Fallback)

For the final deliverable we implemented an ensemble strategy that blends strong in-domain performance with robustness to noisy, OOD text. The system uses **fastText sentence embeddings + logistic regression** as the primary classifier, then **routes high-OOV (out-of-vocabulary) sentences** to a **character n-gram TF-IDF + logistic regression** fallback model that better handles misspellings, code-mixing, and short/noisy inputs. The routing threshold is based on the OOV ratio, with a cutoff of **0.3**; sentences above this threshold are classified by the character n-gram model. The notebook executes this approach end-to-end for five languages (Kazakh, Latvian, Swedish, Yoruba, Urdu) and combines in-domain Wikipedia data with OOD datasets like hate speech and social media for stress testing. This keeps the pipeline lightweight (numpy/pandas/scikit-learn/fastText) while addressing both clean and noisy text regimes.

In the final run, the ensemble achieved **0.9755 accuracy and 0.9754 macro F1** on the in-domain Wikipedia test set, and **0.9764 accuracy with 0.9007 macro F1** on the combined OOD datasets. The routing behavior confirms the intended design: roughly **80.55%** of in-domain sentences and **92.04%** of OOD sentences were sent to the character n-gram fallback, which is more tolerant to high-OOV inputs. This hybrid solution provides a practical deployment path: fastText handles standard cases efficiently, while the fallback protects performance on noisy real-world text without requiring a heavy transformer model.

6 Q1 fastText Experiment

For the Q1 milestone, we also experimented with a fastText classifier as a lightweight, word/character n-gram baseline. We trained a supervised fastText model on the same multilingual Wikipedia snippets, using bag-of-ngrams features to capture language-specific spelling patterns. The model trained quickly and provided an efficient CPU-friendly option, which made it attractive for rapid iteration and ablation comparisons. However, its accuracy lagged behind the character n-gram logistic regression model, especially on short or ambiguous sentences, and it was less stable across closely related language pairs. We therefore treated fastText as a useful benchmark rather than the final recommended approach, but the experiment confirmed that simple n-gram features remain strong signals for language identification and helped validate our final modeling choice.

7 XLM-RoBERTa Experiment and Decision Rationale

We fine-tuned the multilingual transformer XLM-RoBERTa for sequence classification to satisfy the project requirement of a deep learning approach. The model was trained on the same sentence-level Wikipedia snippets, with a standard supervised setup (cross-entropy loss over the ten language labels). XLM-R offered the advantage of contextual understanding, which can, in theory, help when short sentences lack obvious orthographic cues or when borrowed words appear. In qualitative error review, the transformer sometimes corrected cases where the n-gram model over-weighted shared character patterns (e.g., distinguishing closely related Latin-script languages when context words appeared), showing that deep contextual embeddings can resolve subtle distinctions.

However, the transformer did not consistently outperform the character n-gram baseline for this specific task. The gains were marginal on the validation set and not enough to justify the added complexity. The XLM-R model is also significantly heavier: training required GPUs, inference was slower, and deployment would involve larger memory footprints and higher serving costs. From a scalability and cost-efficiency perspective, the n-gram model scales more easily to additional languages and datasets while staying CPU-friendly. From a language-expertise perspective, XLM-R reduces the need for hand-crafted rules but still requires careful hyperparameter tuning and infrastructure, whereas the n-gram model is simpler to retrain and maintain. Given the project's emphasis on practical deployment and resource constraints, we chose the n-gram approach as the final solution, using the XLM-R experiment primarily as a benchmark for deep learning performance and as evidence that simpler models can remain competitive for language identification.

8 Limitations

Despite strong results, the system still struggles when sentences are extremely short, when they contain mostly names or numbers, or when languages share very similar spelling patterns. The n-gram model can also be sensitive to domain shifts (e.g., moving from Wikipedia to social media) and may misclassify code-switched text that mixes languages in a single sentence. The transformer model can help in these cases but is significantly more expensive to run.

9 Possible Next Steps

To improve robustness, we could expand training data to include more informal text and code-switching examples. We could add lightweight confidence scoring and a fallback workflow that routes uncertain predictions to a transformer model or human review. Finally, adding more languages would be straightforward for the n-gram model as long as labeled data is available, making the system scalable for broader multilingual support.